

Balaghi: A Mobile Application for Civic Issue Reporting with a Web-Based Dashboard for Report Management

Academic Project Proposal

Degree: Bachelor of Software Engineering

Date: September 27, 2026

Prepared by:

Ahmed Louay Al-Makki

Ali Maatouq Ghumaid

Mahmoud Ali Al-Hussein

Supervised by:

1. Introduction

Municipal problems such as road potholes, water leaks, and accumulated garbage are often reported through fragmented channels. A useful report must connect a **clear image, reliable coordinates, a category, and a traceable status** so that the responsible team can assess and act on it.

Balaghi proposes one accessible route from the citizen's phone to a municipal dashboard. Visitors can explore a public map without signing in; citizens submit and follow reports; staff review requests before public display, record decisions, and update progress. The first release is limited to one municipality and three issue categories.

2. Scope Of Work

The system covers the core lifecycle of a non-emergency civic report through the following modules:

- **Citizen access:** Phone-number OTP sign-in and a short profile step; no citizen email or password.
- **Report submission:** A new photograph captured with the device camera, GPS coordinates or a manually selected point, an issue category, and an optional description. Gallery selection is unavailable.
- **Public map:** Municipality-reviewed reports, with location markers, category filters, status colours, and public details that exclude reporter identity.
- **My Reports:** A private list of the citizen's submissions and their current states, including a reason when one is rejected.
- **Municipal dashboard:** Admin-issued staff username and password, a review queue for approving or rejecting reports before publication, status updates, and basic statistics.
- **Data and media:** PostgreSQL records, private image storage, and a server API that enforces role-based access.

3. Problem Statement

When reports arrive through calls, messages, or in-person visits, photographs and locations may be missing or separated from the request. Citizens may not know whether the issue was received, while staff lack a single view of requests and their processing history.

The absence of a shared report identifier makes follow-up difficult. A road defect may be described by different street names, while the precise point is unclear. Separate informal messages also make it hard to distinguish a new problem from a repeated question about an existing one.

A useful solution must serve two different views: a public view that helps residents see nearby issues, and a restricted view that gives authorised staff the information needed to respond. Combining these views without exposing a reporter's contact details is a central design requirement.

Balaghi addresses this gap with a location-based workflow. It aims to make submission simple, make public progress visible, and preserve a private operational record for the municipality. Map labels are helpful context, but a report is anchored by coordinates; GPS accuracy and manual correction must still be checked by the user.

4. Objectives

The project has four primary objectives and three supporting objectives that keep the first release feasible for a graduation project.

4.1 Primary Objectives

- **Simple citizen reporting:** Provide an Android flow that captures a new camera photograph, position, category, and optional description in at most four steps, without gallery import.
- **Transparent public map:** Show report locations and states only after municipal review, without exposing the citizen's name, phone, or account identifier.
- **Controlled municipal workflow:** Enable authorised staff to approve or reject reports before public display, record rejection reasons, and track subsequent status changes.
- **Dependable data service:** Store reports, images, and state changes consistently through an API backed by PostgreSQL and Supabase services.

4.2 Secondary Objectives

- **Weak-network resilience:** Keep an unsent report locally and retry it with the same request identifier when connectivity returns.
- **Usability:** Keep the Arabic citizen interface clear, with manual map placement when automatic location is unavailable or inaccurate.
- **Evaluation:** Test the main user journeys, access rules, and map-query performance using a documented demonstration dataset.

5. Literature Review

5.1 Civic Service Requests

Open311 GeoReport v2 describes a common way to submit and track location-based, non-emergency issues such as potholes and street cleaning [1]. It informs Balaghi's separation of service categories, requests, and statuses, without requiring the prototype to claim full Open311 compatibility.

Citizen-generated geographic information can improve the visibility of local conditions, while also requiring attention to data quality and review. Goodchild's discussion of volunteers as geographic observers provides a useful conceptual basis for the citizen contribution model [2].

5.2 Open Mapping and Location

OpenStreetMap supplies a map background, but street-name data may differ from local usage. Balaghi therefore stores latitude and longitude separately from any displayed address, allows a user to move a marker, and treats the address as descriptive text. Map attribution and the chosen tile provider's usage policy remain part of deployment requirements [3, 4].

5.3 Authentication and Privacy

Supabase Auth supports phone-based one-time passwords, while PostgreSQL access rules and server-side checks can limit who sees each record [5, 6]. The OWASP mobile security standard highlights secure storage, authentication, network protection, and privacy controls [7]. Balaghi applies these ideas by keeping reporter identity out of public endpoints and storing sensitive keys only on the server.

6. Project Methodology

A hybrid staged-and-iterative process suits the project. Requirements, data design, and access rules come first; short implementation cycles then deliver demonstrable slices: a public map, citizen login, report submission, private report tracking, and the staff dashboard.

Each cycle ends with a working build and targeted tests. The team will validate phone OTP with development test numbers before using a real SMS provider, then check the route from citizen submission through staff review to public display and status updates. Authentication, data visibility, camera capture, and the weak-network retry path receive explicit test cases.

A submitted report is visible to its author and authorised staff while awaiting review. Staff may approve it for the public map or reject it with a recorded reason. Approval controls visibility; it does not mean that repair work has started. Public statuses remain new, in progress, and resolved.

The first release deliberately excludes live notifications, public comments or voting, automatic image classification, multiple municipalities, and a full iOS release. This controls scope while preserving the essential reporting and follow-up workflow.

Evaluation will use practical acceptance criteria: a complete report in no more than four steps; camera-only capture; correct access for visitors, citizens, and staff; pre-publication review; a recorded status history; and a measured response time for a 500-marker map batch in a stated test environment. Tests will also cover an incorrect OTP, denied location permission, duplicate retry, and report rejection.

The external dependencies with the greatest uncertainty are real SMS delivery to Libyan numbers and map-label quality. The prototype will keep a documented test-number path, and the report form will let the citizen adjust the map pin when the automatic location or street label is unsuitable.

7. Technologies

Layer	Technology	Role
Mobile	Flutter / Dart	Android citizen application and Arabic user interface
Mobile	flutter_map / MapLibre	Interactive public map and report markers
Mobile	Geolocator	Device coordinates with permission handling
Mobile	Secure storage	Local protection for citizen session tokens
Mobile	HTTP client	Requests to the Balaghi API
Backend	Node.js / Express	HTTP API and municipal business rules
Backend	PostgreSQL	Profiles, reports, and status-history records
Backend	Supabase Auth	Citizen OTP and staff account authentication
Backend	Supabase Storage	Private report photographs and short-lived access links
Backend	Sharp	Image processing and metadata removal
Backend	Multer	Multipart image upload handling
Backend	Supabase JS	Server-side authentication and storage integration
Dashboard	React / TypeScript	Planned staff review interface
Mapping	OpenStreetMap data	Street-map context with visible attribution
Mapping	OpenFreeMap	Map style and hosted tile delivery
Testing	Flutter / Node tests	Mobile flow and API behaviour checks
Tools	VS Code / Git	Development, version control, and project documentation

8. Project Timeline

A proposed 22-week plan follows the eight-phase structure of the project. If external SMS setup is delayed, development test numbers keep the authentication flow demonstrable while provider evaluation continues.

Phase	Weeks	Key activities	Milestone
1 Research & Design	1-3	Requirements, interface design, data model, and references	Approved proposal and system design
2 Core Backend	4-7	Database, roles, OTP routes, report API, private images	API and database demonstration
3 Citizen App	8-10	Onboarding, phone login, profile, public map	Working Android navigation
4 Reporting	11-13	Image, GPS/manual point, categories, private list	End-to-end citizen report
5 Staff Dashboard	14-16	Login, pre-publication review, rejection reasons, status changes	Municipal workflow demo
6 Reliability	17-19	Retry queue, access rules, image checks, usability fixes	Resilient integrated build
7 Testing & Docs	20-21	Role tests, performance checks, API and user guide	Documented test results
8 Defense Prep	22	APK, slides, dataset, and rehearsal	Defense-ready submission

9. Expected Outcomes

The project is expected to deliver a testable civic reporting prototype and evidence of its main workflows:

- **An Android citizen application** for camera-and-location reports, viewing reviewed reports on a public map, and personal report tracking.
- **A municipal dashboard** for authorised staff to review reports before publication, record decisions, change status, and view basic counts.
- **A documented service** with controlled access, report history, private media handling, and a repeatable demonstration dataset.

10. Conclusion

Balaghi turns scattered descriptions of civic problems into structured, location-based requests that a municipality can inspect before making them public. The design joins a simple citizen flow with a reviewed public map and a private staff workflow, while keeping the reporter's identity outside public responses.

The proposed Flutter, Express, PostgreSQL, and Supabase stack supports a focused first release for one municipality. Success will be judged by whether a citizen can submit a complete report, staff can move it through a recorded status process, and both sides can see the appropriate view of progress.

The final demonstration will also show the limits of the prototype openly: SMS delivery depends on the configured provider, map labels may require local correction, and the system does not replace emergency-response channels. These boundaries keep the project measurable and honest while leaving a clear route for future expansion.

11. References

1. Open311. (n.d.). *GeoReport v2 specification.* wiki.open311.org/GeoReport_v2/
2. Goodchild, M. F. (2007). Citizens as sensors: The world of volunteered geography. *GeoJournal, 69*, 211-221. doi:10.1007/s10708-007-9111-y
3. OpenStreetMap Foundation. (n.d.). *Copyright and license.* openstreetmap.org/copyright
4. OpenStreetMap Foundation. (n.d.). *Tile usage policy.* operations.osmfoundation.org/policies/tiles/
5. Supabase. (n.d.). *Phone sign-in.* supabase.com/docs/guides/auth/phone-login
6. Supabase. (n.d.). *Row Level Security.* supabase.com/docs/guides/database/postgres/row-level-security
7. OWASP. (n.d.). *Mobile Application Security Verification Standard.* mas.owasp.org/MASVS/
8. Flutter. (n.d.). *Build and release an Android app.* docs.flutter.dev/deployment/android
9. PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation.* postgresql.org/docs/
10. Express. (n.d.). *Express web framework documentation.* expressjs.com/
11. Supabase. (n.d.). *Storage documentation.* supabase.com/docs/guides/storage
12. Baseflow. (n.d.). *Geolocator package documentation.* pub.dev/packages/geolocator
13. Node.js. (n.d.). *Node.js learning resources.* nodejs.org/learn
14. OpenFreeMap. (n.d.). *OpenFreeMap documentation.* openfreemap.org/